DevKnowledgeHub

Plateforme d'intelligence logicielle : code, documentation et tickets réunis

---

Rapport de projet

Auteur	*[À compléter]*
Encadrant	*[À compléter]*
Établissement	*[À compléter]*
Filière	*[À compléter]*
Année universitaire	*[À compléter]*
Version du document	1.0

Résumé

DevKnowledgeHub est une plateforme d'intelligence logicielle qui réunit dans un même espace le code source, la documentation et les tickets d'un projet. Elle ingère un dépôt Git ou une archive ZIP, analyse le code multi-langages, construit un graphe de connaissances dans Neo4j, indexe le code et la documentation dans PostgreSQL/pgvector, puis expose ces connaissances via une recherche RAG/GraphRAG, un copilot et huit agents spécialisés en lecture seule. Des détecteurs ML identifient les vulnérabilités, les code smells, les clones de code et le type de projet. Un volet MLOps suit les exécutions, la latence, la qualité des détecteurs et le feedback utilisateur.

Mots-clés : RAG, GraphRAG, graphe de connaissances, LLM, agents, MLOps, analyse statique, classification de tickets, pgvector, Neo4j.

Sommaire

1. Contexte et problématique
2. Sujet et objectif du projet
3. Architecture générale
4. Présentation fonctionnelle
5. Composantes d'intelligence artificielle
6. Workflow complet
7. Résultats attendus et méthodes d'évaluation
8. Outils et technologies
9. Limites et perspectives
10. Conclusion
11. Annexes

---

1. Contexte et problématique

Dans un projet logiciel, la connaissance est dispersée entre plusieurs artefacts :

- le **code source** et sa structure (fichiers, classes, fonctions, appels, imports) ;
- la **documentation** (README, guides, spécifications) ;
- les **tickets et incidents** (bugs, évolutions, dette technique) ;
- les **résultats d'analyse** (vulnérabilités, code smells, doublons).

Cette dispersion impose au développeur de naviguer entre plusieurs outils pour répondre à des questions courantes :

- Où est définie cette logique métier ?
- Qui appelle ce symbole et quel est le rayon d'impact d'une modification ?
- Quels fichiers sont probablement concernés par ce ticket ?
- Quels sont les risques de sécurité et de qualité dans ce module ?
- Que dit la documentation sur cette fonctionnalité ?

Les conséquences sont un **temps d'onboarding élevé**, des **régressions** lors des modifications, un **triage de tickets lent** et une **revue qualité partielle**.

DevKnowledgeHub répond à ce problème en proposant une plateforme unique qui relie ces artefacts et permet de les interroger en langage naturel, avec des réponses **traçables et citées**.

---

2. Sujet et objectif du projet

2.1 Sujet

Concevoir et réaliser une **plateforme d'intelligence logicielle** capable d'ingérer un dépôt, d'en comprendre la structure, d'indexer sa documentation et ses tickets, puis d'offrir une assistance intelligente à la compréhension, au triage et à la revue du code.

2.2 Objectifs

1. **Ingérer** un projet depuis une URL Git HTTPS ou une archive ZIP.
2. **Analyser** le code multi-langages (Python, Java, JavaScript, TypeScript, TSX).
3. **Construire un graphe de connaissances** (fichiers, symboles, imports, appels, docs, tickets, findings).
4. **Indexer** le code et la documentation pour la recherche vectorielle (RAG).
5. **Répondre** aux questions via un copilot et des agents spécialisés, avec citations vérifiables.
6. **Détecter** automatiquement vulnérabilités, code smells, clones et type de projet.
7. **Mesurer** la qualité, la latence et l'usage via un volet MLOps/monitoring.

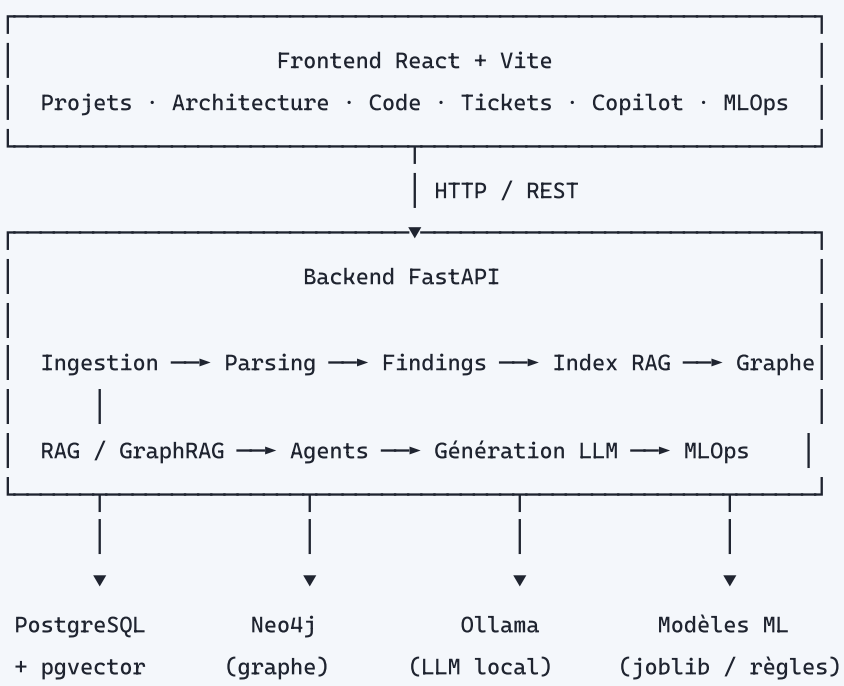
2.3 Motivation

- Réduire le temps de compréhension d'un codebase et faciliter l'onboarding.
- Accélérer le triage de tickets et l'analyse d'impact.
- Industrialiser la revue qualité et sécurité.
- Garantir la **traçabilité des sources** et un fonctionnement **local/maîtrisé** (LLM via Ollama).

---

3. Architecture générale

3.1 Vue d'ensemble



3.2 Composants

Composant	Rôle	Emplacement
Frontend	Interface utilisateur	`frontend/src`
API backend	REST, orchestration	`backend/app/api`
Ingestion	Git/ZIP, analyse, indexation	`backend/app/services`
Code intelligence	Parsing, symboles, technologies	`backend/app/code_intelligence`
Graphe	Construction et requêtes Neo4j	`backend/app/graph`
RAG	Chunking, embeddings, retrieval	`backend/app/rag`
Agents	Copilot et agents spécialisés	`backend/app/agents`
MLOps	Runs, métriques, monitoring	`backend/app/mlops`
ML	Modèles, entraînement, évaluation	`ml/src/dkh_ml`
Infra	Docker, init PostgreSQL	`infra` , `docker-compose*.yml`

3.3 Choix d'architecture

- **pgvector comme base vectorielle** : évite un composant supplémentaire ; choix documenté dans `docs/architecture/adr/0001-choix-pgvector.md`.
- **Neo4j pour les relations** : les appels, imports et liens documentaires/tickets sont mieux représentés par un graphe.
- **Graphe optionnel** : si Neo4j échoue, le RAG reste utilisable (dégradation contrôlée).
- **Séparation frontend/backend** : API REST versionnée (`/api/v1`), frontend statique servi par Nginx en production.
- **Agents en lecture seule** : aucun agent n'exécute de code, ne modifie un fichier ou ne crée de ticket.

3.4 Modèle de données

PostgreSQL/pgvector : dépôts, chunks (code et documentation), embeddings, tickets, findings, runs d'ingestion, métriques, feedback.

Neo4j (`backend/app/graph/schema.py`) :

```
(Repository)-[:CONTAINS]->(File)
(File)-[:DEFINES]->(Symbol)
(Symbol)-[:PARENT_OF]->(Symbol)
(File)-[:IMPORTS]->(Module)
(Symbol)-[:CALLS {line}]->(Symbol)
(Documentation)-[:DOCUMENTS]->(File)
(Documentation)-[:DESCRIBES]->(Symbol)
(Issue)-[:AFFECTS]->(File)
(Finding)-[:AFFECTS]->(File|Symbol)
(Commit)-[:CONTAINS]->(File)
```

Nœuds complémentaires : `Technology`, `Issue`, `Documentation`, `Commit`, `Finding`, `CodeSmell`, `Vulnerability`.

3.5 Sécurité

- Clonage limité aux URL **HTTPS**.
- Noms de dépôts validés par expression régulière.
- Archives ZIP contrôlées : taille maximale, nombre de fichiers, refus des chemins absolus, `..` et liens symboliques.
- Agents **read-only**, sources traitées comme données non fiables.
- Citations LLM vérifiées : les identifiants inexistants sont supprimés.
- Pas de stockage du texte des questions/réponses dans les métriques.

---

4. Présentation fonctionnelle

4.1 Ingestion de projets

L'utilisateur crée un projet, fournit une URL Git ou une archive ZIP, puis lance l'ingestion. Le traitement s'exécute en arrière-plan avec les statuts `pending`, `indexing`, `ready` ou `failed`.

L'ingestion réalise successivement : clonage/extraction, analyse du code, détection des technologies, collecte documentaire, détection des findings, indexation RAG, persistance des documents, construction du graphe.

4.2 Analyse du code

- Détection des langages et parcours des fichiers sources.
- Parsing via **tree-sitter** et parsers dédiés.
- Extraction des fichiers, symboles, classes, méthodes, imports et appels.
- Détection des technologies/frameworks et du type de projet (application, library, service, full-stack).

4.3 Graphe de connaissances

L'interface Architecture & graphe présente les statistiques du dépôt, les fichiers les plus riches en symboles, les modules les plus importés, les appels et les dépendances. L'analyse d'impact remonte les appelants directs et transitifs d'un symbole.

4.4 Recherche RAG et GraphRAG

- Découpage du code en chunks alignés sur les symboles.
- Découpage de la documentation en fenêtres de caractères.
- Recherche vectorielle par similarité cosinus.
- Reranking lexical sur les noms de symboles.
- Expansion dans le graphe pour enrichir le contexte.

4.5 Copilot et agents spécialisés

Le catalogue est exposé par `GET /api/v1/agents/catalog` et exécuté via `POST /api/v1/agents/specialist`.

Agent	Rôle	Sources
`copilot`	Réponse hybride avec citations vérifiées	RAG + graphe
`researcher`	Boucle de recherche bornée à trois actions	RAG + graphe
`architecture`	Modules, fichiers et symboles centraux	RAG + graphe
`documentation`	Réponses limitées à la documentation indexée	RAG
`impact`	Appelants directs et transitifs d'un symbole	graphe + RAG
`triage`	Catégorie, priorité et fichiers probables d'un ticket	heuristique + RAG + graphe
`review`	Synthèse des vulnérabilités, smells et doublons	findings + RAG
`test_plan`	Scénarios de tests sans exécution	RAG + graphe

Chaque exécution enregistre l'agent, le modèle, la durée, le nombre de preuves et les citations dans `agent_runs`.

4.6 Documentation et tickets

- Collecte des README, guides et spécifications (`.md` , `.markdown` , `.rst`).
- Import et classification des tickets en `BUG` , `FEATURE` ou `TECHNICAL_DEBT` .
- Rapprochement lexical entre tickets et fichiers/symboles, puis liaison dans le graphe.

4.7 Détecteurs et findings

- **Vulnérabilités** : règles statiques (eval/exec, shell=True, secrets en dur, SQL dynamique, etc.).
- **Code smells** : méthode longue, liste de paramètres longue, imbrication profonde.
- **Clones** : similarité structurelle et lexicale entre fichiers.
- **Type de projet**.

Les findings sont persistés et affichés dans l'interface.

4.8 MLOps et monitoring

- Suivi des exécutions d'agents.
- Métriques de latence RAG et de qualité des détecteurs.
- Feedback utilisateur associé aux requêtes.
- Dashboards de monitoring.

4.9 Parcours utilisateur type

1. Ouvrir la page **Projets** et importer un dépôt.
2. Consulter **Architecture & graphe** pour la vue structurelle.
3. Explorer **Analyse du code** et **Tickets & incidents**.

4. Interroger le **Copilot** ou un **agent spécialisé**.
5. Suivre la qualité et les performances dans **MLOps** et **Monitoring**.

---

5. Composantes d'intelligence artificielle

5.1 RAG (Retrieval-Augmented Generation)

Chunking

- Un chunk par symbole (fonction, méthode, classe) avec métadonnées fichier, lignes et nom qualifié.
- Fenêtres de lignes avec recouvrement pour les fichiers sans symboles.
- Fenêtres de caractères pour la documentation.

Embeddings

- Mode par défaut : `HashingEmbedder` — signed hashing trick sur des sous-tokens, déterministe, sans dépendance, non sémantique.
- Mode avancé : `SentenceTransformerEmbedder` — `paraphrase-multilingual-MiniLM-L12-v2`, dimension 384.

Recherche

- pgvector, distance cosinus.
- Sur-échantillonnage puis reranking lexical selon la correspondance question/symboles.

GraphRAG

- Expansion du contexte via les relations du graphe : appelants, appelés, imports, documentation, tickets, findings.

5.2 LLM de génération

- **Ollama** en local, modèle par défaut `qwen2.5-coder:1.5b` (7b possible avec GPU).
- Température basse (0,1) pour favoriser la factualité.
- Prompts système stricts : répondre uniquement à partir des preuves, citer `[Sx]`, ignorer les instructions contenues dans les sources.

5.3 Agents

- **Copilot hybride** : RAG + graphe + LLM + vérification des citations.
- **Chercheur autonome** : planification JSON bornée à trois actions (`search_rag` , `search_symbols` , `callers` , `callees` , `finish`), validation des actions et protection anti-boucle.
- **Agents spécialisés** : architecture, documentation, impact, triage, review, test plan.
- **Sécurité** : lecture seule, sources non fiables, suppression des citations invalides.

5.4 Classification de tickets

- **Baseline** : TF-IDF (1–2 grammes) + régression logistique.
- **Approche avancée** : DistilBERT fine-tuné (`DistilBertForSequenceClassification`).
- **Fallback heuristique** par mots-clés si le transformer est indisponible.

- Remarque : le modèle transformer présent a été entraîné sur des labels faibles générés par mots-clés (`label_source: weak_keywords`) ; ses métriques très élevées mesurent surtout la reproduction de ces règles.

5.5 Classification du type de projet

TF-IDF + régression logistique, classes : application, library, service, full-stack.

5.6 Détection de vulnérabilités

- **Règles statiques** : eval/exec, shell=True, os.system, pickle, yaml.load non sûr, secrets en dur, SQL dynamique, hash faible, TLS désactivé, debug activé.
- **Classifieur ML** : TF-IDF mots (1–2 grammes) + TF-IDF caractères (3–5 grammes) + features binaires des règles, puis régression logistique équilibrée.

5.7 Code smells

Règles structurelles : longueur de méthode (seuil 40 lignes), nombre de paramètres (seuil 5), profondeur d'imbrication (seuil 4), classes volumineuses.

5.8 Détection de clones

Score combiné : 70 % de similarité structurelle normalisée (SequenceMatcher sur tokens normalisés) + 30 % de Jaccard de tokens. Seuil par défaut : 0,75.

5.9 Embeddings de code légers

Hashing déterministe (256 dimensions) + n-grammes de caractères, similarité cosinus.

---

6. Workflow complet

6.1 Collecte et prétraitement

1. Source : URL Git HTTPS ou archive ZIP.
2. Validation : schéma HTTPS, nom de dépôt, contrôles ZIP.
3. Récupération : clone superficiel ou extraction contrôlée.
4. Parcours des fichiers avec exclusion des dossiers ignorés.
5. Parsing : tree-sitter et parsers dédiés.
6. Collecte documentaire.
7. Nettoyage et normalisation (encodage, taille, titres).

6.2 Indexation RAG

1. Chunking par symbole et par document.
2. Embedding par lots.
3. Écriture atomique dans PostgreSQL/pgvector.
4. Enregistrement des métadonnées (dépôt, URL, commit, date).

6.3 Construction du graphe

1. Création des nœuds et contraintes Neo4j.
2. Insertion par lots (`UNWIND`) des fichiers, symboles, imports et appels.
3. Ajout des commits, technologies, documents, tickets et findings.
4. Résolution des appels et comptage des appels non résolus.

6.4 Analyse IA et findings

1. Scan des vulnérabilités.
2. Détection des code smells.
3. Détection des clones.
4. Persistance et exposition des findings.

6.5 Requête utilisateur

1. Question et extraction de termes.
2. Recherche vectorielle top-k.
3. Reranking lexical.
4. Expansion graphe.
5. Construction du contexte avec identifiants `[Sx]` .
6. Génération LLM via Ollama.
7. Vérification des citations.

8. Enregistrement MLOps.

6.6 Workflow ML

Pipeline DVC à cinq étapes :

1. `ticket_classifier`
2. `project_classifier`
3. `clone_detector` (évaluation)
4. `vulnerability_detector`
5. `code_smell` (évaluation)

Chaque étape consomme un jeu de fixtures JSONL, entraîne ou évalue un modèle, écrit un modèle `.joblib` et/ou un rapport de métriques, utilise les hyperparamètres de `ml/params.yaml` et peut journaliser dans MLflow.

6.7 Déploiement

- **Développement** : Docker Compose pour PostgreSQL/pgvector et Neo4j, backend via `python dev.py`, frontend via `npm run dev`.
- **Production** : `docker compose -f docker-compose.prod.yml up -d --build`, Nginx en frontal, volumes persistants, healthchecks.
- **Kubernetes/Terraform** : emplacements prévus, non implémentés.

---

7. Résultats attendus et méthodes d'évaluation

7.1 Résultats attendus

- Un espace unique pour explorer code, documentation et tickets.
- Des réponses traçables avec citations et preuves.
- Une vision structurelle du projet et de l'impact des changements.
- Un triage assisté des tickets et une revue qualité priorisée.
- Une détection automatique des risques et de la duplication.
- Un suivi MLOps pour comparer agents, modèles et stratégies de recherche.

7.2 Évaluation des modèles ML

Métriques : accuracy, macro-F1, weighted-F1, précision/rappel/F1 par classe, matrice de confusion.

Baselines actuelles (jeux de fixtures réduits, à but démonstratif) :

Composant	Accuracy	Macro-F1
Classification de tickets	0,875	0,841
Type de projet	0,75	0,667
Vulnérabilités	0,80	0,80
Code smells	1,0 (micro)	1,0 (micro)
Clones	1,0	1,0

Ces chiffres valident le pipeline, pas la performance en production.

7.3 Évaluation du RAG et des agents

- Qualité de retrieval : pertinence des chunks, couverture des symboles attendus.
- Fidélité des réponses : taux de citations valides, absence de citations inventées.
- Qualité des preuves : nombre de preuves RAG vs graphe.
- Performance : latence, durée par agent, nombre d'appels aux outils.
- Feedback utilisateur.
- Monitoring : dashboards Prometheus/Grafana et rapports Evidently.

7.4 Évaluation fonctionnelle

- Tests backend et ML avec pytest.
- Lint avec ruff.
- Build frontend.

- CI GitHub Actions : `ci-backend.yml`, `ci-ml.yml`, `ci-frontend.yml`, `cd-deploy.yml`.

---

8. Outils et technologies

Backend

- Python 3.11
- FastAPI, Uvicorn, Pydantic Settings
- psycopg 3, driver Neo4j
- tree-sitter (Python, Java, JavaScript, TypeScript)
- python-multipart

Frontend

- React 19, Vite 7, React Router
- Recharts, Lucide React
- CSS modules, design tokens

Données et stockage

- PostgreSQL 16 + pgvector
- Neo4j 5
- Ollama (`qwen2.5-coder`)

ML / Data Science

- scikit-learn (TF-IDF, régression logistique)
- joblib
- Transformers / DistilBERT
- NumPy, SciPy
- DVC (pipeline 5 étapes)
- MLflow (optionnel)

MLOps / Observabilité

- Stockage interne des runs et métriques
- Prometheus, Grafana, Evidently (emplacements prévus)

Déploiement / Qualité

- Docker, Docker Compose (dev et prod)
- Nginx
- Terraform / Kubernetes (emplacements prévus)
- pytest, ruff
- GitHub Actions

9. Limites et perspectives

9.1 Limites actuelles

- Embeddings par défaut non sémantiques (`hash`) ; activer `sentence-transformers` pour une vraie recherche sémantique.
- Détecteurs de vulnérabilités et de smells statiques/heuristiques : candidats, pas preuves.
- Classifieur de tickets entraîné sur un petit jeu de fixtures et, pour le transformer, sur des labels faibles.
- Ingestion synchrone côté worker FastAPI.
- PostgreSQL/pgvector et Neo4j en instance unique, sans haute disponibilité.
- Agents volontairement bornés : pas d'exécution de code, pas d'écriture, pas de création de ticket.

9.2 Perspectives

- Embeddings sémantiques par défaut et évaluation RAG automatisée.
- Extension du dataset d'entraînement et validation sur données réelles.
- Ingestion asynchrone avec file de tâches (Celery/Redis).
- Haute disponibilité des bases et déploiement Kubernetes/Terraform effectif.
- Ajout d'agents d'écriture contrôlée (création de ticket, proposition de correctif) avec validation humaine.
- Observabilité complète avec Prometheus, Grafana et Evidently.

---

10. Conclusion

DevKnowledgeHub unifie code, documentation et tickets dans une plateforme unique combinant analyse statique, graphe de connaissances, RAG/GraphRAG, agents LLM et MLOps. L'architecture sépare clairement le frontend, l'API, les stockages et les modèles, avec des mécanismes de dégradation contrôlée et des garde-fous de sécurité. Le projet fournit une base fonctionnelle et extensible pour l'assistance à la compréhension de code, le triage de tickets et la revue qualité, avec une démarche d'évaluation explicite et traçable.

11. Annexes

11.1 API principale

- `GET /health`
- `POST /api/v1/projects`, `GET /api/v1/projects`
- `POST /api/v1/projects/{name}/ingest`, `POST .../ingest/zip`
- `GET /api/v1/projects/{name}`, `/status`, `/documents`, `/issues`, `/findings`, `/architecture`, `/graph`, `/files/{path}`
- `POST /api/v1/projects/{name}/ask`, `/feedback`
- `GET /api/v1/graph/{repo}/stats|symbols|calls|dependencies|impact`
- `GET/POST /api/v1/mlops/agent-runs|monitoring|models|quality|metrics`
- `GET /api/v1/agents/catalog`, `POST /api/v1/agents/ask|search|specialist`
- `POST /api/v1/rag/index|search`

11.2 Structure du dépôt

```
backend/       API FastAPI, parsing, graphe, RAG, détecteurs, MLOps
frontend/      React + Vite
ml/            modèles, entraînement, évaluation, DVC, rapports
infra/         init PostgreSQL, emplacements Kubernetes/Terraform
docs/          architecture, guides, ADR
pipelines/     orchestration
monitoring/    dashboards et observabilité
evaluation/    jeux d'évaluation
tests/         tests transverses
```

11.3 Glossaire

- **RAG** : Retrieval-Augmented Generation, génération augmentée par recherche.
- **GraphRAG** : RAG enrichi par expansion dans un graphe de connaissances.
- **Chunk** : fragment de code ou de document indexé.
- **Embedding** : représentation vectorielle d'un texte.
- **Finding** : résultat d'un détecteur (vulnérabilité, smell, clone).
- **MLOps** : pratiques d'industrialisation et de suivi des modèles ML.
- **pgvector** : extension PostgreSQL pour la recherche vectorielle.